

I. 전체 시스템 아키텍처 및 Root 구조

Shopping Mall AI - 파일별 역할 및 책임 상세 분석

I 1. 전체 디렉토리 구조 (Vertical Slice)

```
shopping_mall_ai/
|
├─ 【Root 설정 파일】
|   ├─ app.py           # 🎯 Streamlit 메인 진입점
|   ├─ config.py        # ⚙️ 환경 설정 (API, 경로)
|   └─ ...
|
├─ 【features/ - 기능별 모듈】
|   ├─ chat/            # 💬 대화형 챗봇 (UI + 로직)
|   ├─ rag/             # 🔍 RAG 파이프라인
|   ├─ dashboard/       # 📊 분석 대시보드
|   ├─ settings/        # ⚙️ 관리자 설정
|   └─ shared/          # 🔗 공유 유틸리티
|
├─ sample_data/        # 📄 정책 문서 원본 (.txt)
└─ data/               # 🗄️ 생성된 인덱스 & DB
    ├─ faiss_index/
    │   ├─ index.faiss
    │   └─ chunks.json
    └─ chat.db          # SQLite 대화 이력
```

I 2. Root 레벨 설정 파일

파일	역할	상세 책임
app.py	🎯 진입점	Streamlit 멀티페이지 라우팅 및 세션 상태 초기화 관리
config.py	⚙️ 설정	API 키, LLM 모델 파라미터, DB/인덱스 경로 중앙 통제
requirements.txt	📦 패키지	구동에 필요한 의존성 라이브러리 및 버전 명시

I 3. 공통 핵심 파라미터 (config.py)

- **LLM_PROVIDER**: "gemini" (답변 생성 모델)
- **GENERATION_MODEL**: "gemini-3.5-flash-lite"
- **EMBEDDING_PROVIDER**: "ollama" (벡터 변환)
- **OLLAMA_EMBEDDING_MODEL**: "nomic-embed-text" (768d)
- **CHUNK_SIZE**: 500 / **OVERLAP**: 100
- **TOP_K**: 5 (유사도 검색 결과 개수)

II. 챗봇 서비스 모듈 분석

대화 처리, 체인 오케스트레이션 및 프롬프트 관리 구조

I 🤖 chatbot_service.py (메인 서비스 로직)

핵심 역할: UI와 비즈니스 로직을 연결하는 컨트롤러

- `process_message()`: 사용자 질문 처리 및 응답 반환
- `_classify_category()`: 질문 내 키워드 기반 카테고리 자동 분류
- RAG 체인(`rag_chain.invoke`) 연동 및 호출
- 대화 내역 및 사용자 평가(👍 😐 👎) 데이터베이스 영속화

I 🧠 rag_chain.py (LCEL 파이프라인)

구조: {context, question} → Prompt → LLM → OutputParser

- LangChain의 LCEL(| 연산자) 문법을 활용한 파이프라인 구축
- Retriever를 통한 동적 컨텍스트(문서 조항) 주입 조절
- StrOutputParser를 활용한 LLM 응답 텍스트 정제 추출

I 🎨 chatbot_ui.py (화면 렌더링)

핵심 역할: 사용자 상호작용 및 실시간 화면 업데이트

- `st.chat_input` 및 `st.chat_message` 컴포넌트 렌더링
- 스트리밍 방식의 AI 응답 표시 지원
- 대화 히스토리 및 피드백 버튼 UI 표시
- `st.session_state` 기반 상태 동기화

I 🗨️ prompts.py (프롬프트 템플릿)

핵심 역할: 카테고리별 특화 프롬프트 디렉터리 관리

- **GENERAL_PROMPT:** 기본 응대 및 범용 지침
- **SHIPPING_PROMPT:** 출고 시간, 지역 운임 포커싱
- **RETURN_PROMPT / PAYMENT_PROMPT:** 환불 기한, 수단별 취소 기간 등 집중 조명

III. RAG 지식 검색 파이프라인 분석

문서 전처리부터 임베딩, 벡터 색인 및 유사도 검색 프로세스

document_loader.py (문서 수집)

- sample_data/ 디렉토리 내 .txt 파일 일괄 스캔
- UTF-8 인코딩 텍스트 추출 및 파일명 메타데이터 결합
- {"filename": "...", "content": "..."} 형태의 딕셔너리 리스트 반환

text_splitter.py (청크 분할)

- RecursiveCharacterTextSplitter 활용 구조적 분할
- 분할 구분자(["\n\n", "\n", ".", ", ", ""']) 우선순위 적용
- 500자(CHUNK_SIZE) 제한, 100자(OVERLAP) 중복 허용으로 문맥 보존

embedding_generator.py (벡터 변환)

- Ollama API 기반 nomic-embed-text 모델 연동
- 분할된 청크 텍스트를 768차원 고밀도 벡터(Dense Vector)로 변환
- 속도 최적화를 위한 임베딩 배치(Batch) 처리 구조 설계

faiss_indexer.py (벡터 색인 저장)

- FAISS IndexFlatL2 (유클리디안 거리 연산) 인덱스 생성
- 생성된 임베딩 배열을 인덱스 메모리에 등재
- index.faiss(벡터), chunks.json(원본 텍스트) 분리 영속화

retriever.py (유사도 검색 엔진)

검색 실행 프로세스 (Runtime Retrieval)

1. **질문 벡터화**: 사용자의 질문(Query)을 동일하게 768차원 임베딩으로 변환
2. **FAISS 탐색**: index.search()를 호출하여 가장 가까운 벡터 Top-K 추출
3. **데이터 결합**: 도출된 인덱스 번호를 바탕으로 chunks.json에서 원본 텍스트(문맥) 복원
4. **반환**: 거리(Distance) 스코어와 문서 내용을 포맷팅하여 LCEL 체인으로 전달

IV. 대시보드, 관리자 설정 및 공유(Shared) 유틸리티

운영 관리를 위한 부가 기능 및 시스템 전역 의존성 주입 계층

📊 features/dashboard/analytics_ui.py

상담 통계 및 차트 시각화 (관리자용)

- 일자별 상담 트렌드 라인 차트 구현
- 긍정/부정/보통 평가 분포 파이 차트 및 카테고리 통계 제공
- 부정 평가(-1점) 이력 역추적 및 원인 분석 테이블 표시

⚙️ features/settings/settings_ui.py

지식 베이스(Vector DB) 제어 패널

- 정책 파일 수동 업로드 UI 및 로딩 상태 제어
- FAISSIndexBuilder 원클릭 재구성 및 기존 캐시 초기화
- 생성 진행률(Progress) 실시간 피드백 렌더링

🔗 features/shared/ (공유 의존성 계층)

파일	주요 클래스/메서드	핵심 책임(Responsibility)
api.py	create_llm_api() create_embedding_api()	Factory Pattern 적용: 환경 설정에 따라 Gemini 또는 Ollama 클라이언트 객체를 동적으로 생성하여 추상화된 프로바이더 인터페이스 반환
db.py	save_chat() rate_message() get_statistics()	SQLite 영속성 관리: chats 테이블 스키마 정의. 질문/응답/카테고리/평점/타임스탬프 로깅 및 통계 집계 쿼리 실행
state_manager.py	docs_loaded messages	Session State 제어: Streamlit의 고유한 리렌더링 사이클에서 문서 로딩 상태, 챗봇 대화 배열 등을 안전하게 전역 보존

V. 전체 시스템 데이터 흐름 및 구동 순서

사용자 입력부터 응답 반환, 통계 축적까지의 라이프사이클

🔄 End-to-End 데이터 흐름도

```
graph TD
    A["【사용자 입력】 \"배송기간은 얼마나 걸리나요?\""] --> B["【chatbot_ui.py】 사용자 채팅 인풋(st.chat_input) 수집"]
    B --> C["【chatbot_service.py】 process_message() 호출 및 의도 분류 (Category: SHIPPING)"]
    C --> D["【rag_chain.py】 invoke() 실행"]
    D --> E["└ retriever.retrieve() → 쿼리 벡터화 → FAISS 유사도 탐색 → chunks 도출"]
    D --> F["└ 분류된 카테고리의 PromptTemplate 에 [컨텍스트 + 질문] 매핑 주입"]
    D --> G["└ LLM API (Gemini-3.5-flash) 네트워크 호출 (api.py)"]
    D --> H["└ StrOutputParser 로 텍스트 추출"]
    E --> I["【chatbot_service.py】 최종 응답을 db.save_chat()을 통해 SQLite 에 로깅"]
    F --> I
    G --> I
    H --> I
    I --> J["【chatbot_ui.py】 응답 내용 화면 렌더링 및 (👍/😐/👎) 피드백 버튼 활성화"]
    J --> K["【db.py / analytics_ui.py】 사용자가 버튼 클릭 시 DB에 반영 및 대시보드 통계 즉각 업데이트"]
```

🎯 모듈별 라이프사이클 (실행 순서)

1 초기화 (Settings)

1. DocumentLoader (정책 원본 로드)
2. TextSplitter (500자 청킹)
3. EmbeddingGenerator (벡터 생성)
4. FAISSIndexBuilder (저장)

2 대화 처리 (Chat)

1. 질문 입력 대기
2. RAGChain 구동 (검색+추론)
3. SQLite 이력 저장
4. 스트리밍 렌더링 및 피드백

3 운영 분석 (Dashboard)

1. DB에서 최신 대화 로그 Read
2. 카테고리/만족도 통계 연산
3. matplotlib/plotly 기반 시각화
4. 오답 모니터링 관리